

Sri Lanka Institute of Information Technology



Web-based Employee Management System

2025-Y2-S1-MLB-B10G2-01

Group ID	Student Name
IT24102622	Amarasinghe V. A
IT24102614	Kahadawala D. C
IT24102607	Irushi Umada M. M .A
IT24102627	Fernando W. D. N
IT24102615	Mamalgaha I. G. W. S
IT24102669	Sathsarani M. B. N. K

Design Pattern(s) Used

- Strategy Design Pattern.

Justification of Design Pattern

- We applied Strategy design pattern for **notification** sending component in our system, to handle different types of notification delivering methods.
- Notification handle has multiple delivery methods like Email, SMS, Push and In-App, use of this design pattern makes it flexible and maintainable.
- Each delivery type has its own DeliveryHandler, allowing NotificationClient to send notifications flexibly without handling delivery specifics directly.
- This design reduces conditional logic and also makes it easy to add new delivery methods without modifying existing code.

Screenshots of Implementation

Strategy Interface (DeliveryHandler)

```
2
3 > import ...
4
5
6 public interface DeliveryHandler { 7 usages 4 implementations
7     boolean canDeliver(Notification.DeliveryMethod method); 1 usage 4 implementations
8     void deliver(Notification notification, UserAccount recipient); 1 usage 4 implementations
9     String getHandlerName(); no usages 4 implementations
10 }
11
```

Concrete Strategies

1.(EmailDeliveryHandler)

```
6
7  @Service
8  public class EmailDeliveryHandler implements DeliveryHandler {
9
10     @Override 1 usage
11     public boolean canDeliver(Notification.DeliveryMethod method) {
12         return method == Notification.DeliveryMethod.EMAIL;
13     }
14
15     @Override 1 usage
16     public void deliver(Notification notification, UserAccount recipient) {
17         // In a real application, integrate with email service (SendGrid, AWS SES, etc.)
18         System.out.println("Email notification sent:");
19         System.out.println("To: " + recipient.getUsername() + "@company.com");
20         System.out.println("Subject: " + notification.getTitle());
21         System.out.println("Body: " + notification.getMessage());
22         System.out.println("Priority: " + notification.getPriority().getDisplayName());
23         System.out.println("Alert Type: " + notification.getAlertType().getDisplayName());
24     }
25
26     @Override no usages
27     public String getHandlerName() { return "Email Delivery"; }
28 }
```

2.(InAppDeliveryHandler)

```
@Service
public class InAppDeliveryHandler implements DeliveryHandler {

    private final NotificationRepository notificationRepository; 1 usage

    public InAppDeliveryHandler(NotificationRepository notificationRepository) {
        this.notificationRepository = notificationRepository;
    }

    @Override 1 usage
    public boolean canDeliver(Notification.DeliveryMethod method) {
        return method == Notification.DeliveryMethod.IN_APP;
    }

    @Override 1 usage
    public void deliver(Notification notification, UserAccount recipient) {
        // In-app notifications are already saved to database
        // This handler is called for additional processing if needed
        System.out.println("In-app notification delivered: " + notification.getTitle() + " to " + recipient.getUsername());
    }

    @Override no usages
    public String getHandlerName() { return "In-App Delivery"; }
}
```

3.(PushDeliveryHandler)

```
@Service
public class PushDeliveryHandler implements DeliveryHandler {

    @Override 1 usage
    public boolean canDeliver(Notification.DeliveryMethod method) { return method == Notification.DeliveryMethod.PUSH; }

    @Override 1 usage
    public void deliver(Notification notification, UserAccount recipient) {
        // In a real application, integrate with push notification service (FCM, APNS, etc.)
        System.out.println("Push notification sent:");
        System.out.println("To: " + recipient.getUsername());
        System.out.println("Title: " + notification.getTitle());
        System.out.println("Body: " + notification.getMessage());
        System.out.println("Priority: " + notification.getPriority().getDisplayName());
        System.out.println("Alert Type: " + notification.getAlertType().getDisplayName());
    }

    @Override no usages
    public String getHandlerName() { return "Push Delivery"; }
}
```

4.(SmsDeliveryHandler)

```
@Service
public class SmsDeliveryHandler implements DeliveryHandler {

    @Override 1 usage
    public boolean canDeliver(Notification.DeliveryMethod method) { return method == Notification.DeliveryMethod.SMS; }

    @Override 1 usage
    public void deliver(Notification notification, UserAccount recipient) {
        // In a real application, integrate with SMS service (Twilio, AWS SNS, etc.)
        System.out.println("SMS notification sent:");
        System.out.println("To: +1234567890"); // Would use recipient's phone number
        System.out.println("Message: " + notification.getTitle() + " - " + notification.getMessage());
        System.out.println("Priority: " + notification.getPriority().getDisplayName());
    }

    @Override no usages
    public String getHandlerName() { return "SMS Delivery"; }
}
```

Client (NotificationClient)

```
private void deliverNotification(Notification notification, UserAccount recipient) { 1 usage
    NotificationSettings settings = getOrCreateSettings(recipient);

    // Check if delivery is enabled for this alert type and method
    if (settings.isDeliveryEnabled(notification.getAlertType(), notification.getDeliveryMethod())) {
        deliveryHandlers.stream() Stream<DeliveryHandler>
            .filter( DeliveryHandler handler -> handler.canDeliver(notification.getDeliveryMethod()))
            .findFirst() Optional<DeliveryHandler>
            .ifPresent( DeliveryHandler handler -> handler.deliver(notification, recipient));
    }
}

private NotificationSettings getOrCreateSettings(UserAccount user) { 2 usages
    return settingsRepository.findByUser(user)
        .orElseGet(() -> {
            NotificationSettings settings = new NotificationSettings(user);
            return settingsRepository.save(settings);
        });
}
```

